PROJECT 27 - ENSURING OPERATIONAL EXCELLENCE WITH IOT MONITORING AT EPICREADS

Cloud Computing Capstone Project

[Course Code - _____]

Submitted in partial fulfillment of the requirements for the degree of
Master of Technology in Integrated Computer Science Engineering (Specialization in Data Science)
in
School of Computer Science Engineering and Information Systems

by

AKKI PRAJWAL
24MID0230

Under the guidance of
Dr. Sivakumar
Associate Professor, SCORE
School of Computer Science Engineering and Information Systems
VIT, Vellore

October, 2026

DECLARATION

I hereby declare that the Cloud Computing Capstone Project entitled "Project 27 - Ensuring Operational Excellence with IoT Monitoring at EpicReads" submitted to VIT, Vellore is a record of bonafide work carried out by me. The work embodies results of my own effort, supported where stated by the project repository, its dataset engineering pipeline and its automated verification scripts.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institution or any other institution or university.

Place: Vellore Date: _____

Akki Prajwal 24MID0230
Signature of the Candidate

CERTIFICATE

This is to certify that the Cloud Computing Capstone Project entitled "Project 27 - Ensuring Operational Excellence with IoT Monitoring at EpicReads" submitted by Akki Prajwal (24MID0230), in partial fulfillment of the requirements for the award of the degree of Master of Technology in Integrated Computer Science Engineering with specialization in Data Science, VIT, Vellore, is a record of bonafide work carried out by him under my supervision and guidance.

The contents of this report have not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institution or any other institution or university. The report attests to the student's work in this capstone project.

Signature of the VIT SCORE Supervisor

Dr. Sivakumar
Associate Professor, SCORE

Internal Examiner External Examiner

Head of the Department Department of Software and Systems Engineering

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Dr. Sivakumar, Associate Professor, SCORE, VIT, Vellore, for his academic guidance and support during the project. His feedback on the analytics architecture, the anomaly-detection design and the verification standards used in this report shaped the work substantially.

I thank the School of Computer Science Engineering and Information Systems, VIT, for the academic environment, the laboratory resources and the course framework that made this capstone possible.

I am grateful to my family, teachers and friends for their encouragement throughout my studies and project work. The open-source software ecosystem on which this project is built (React, TypeScript, Chart.js, Node.js, pdfkit, axios) and the maintainers of the libraries used made this work executable, and I acknowledge all those who contributed indirectly.

Akki Prajwal
24MID0230

EXECUTIVE SUMMARY

Industrial print farms fail quietly: a bearing overheats, ink runs low in the middle of a job, and a jammed feeder is noticed only when the delivery is already late. This project implements and measures a complete IoT monitoring system for the fleet of eight industrial printers at EpicReads - machines EPR-001 through EPR-008 - using a synthetic-but-realistic operational dataset of 6,912 telemetry readings collected at a 15-minute cadence over three calendar days (2026-10-15 00:00 to 2026-10-17 23:55), across seven operational sensors (temperature, humidity, ink level, paper stock, motor RPM, vibration and power) and the pages-per-minute and paper-jam channels.

A Node.js analytics backend computes the fleet key performance indicators, per-printer distributions, hourly and daily trends, and per-printer threshold rules derived from printers.csv, and serves them over a REST API; a React + TypeScript + Chart.js dashboard renders five sections (Overview, Anomaly Detection, Telemetry Explorer, Prediction and Live Monitor). The anomaly layer follows a deliberate design: raw threshold-breach detection on each reading, a weighted risk score for live sensor inputs, and a three-consecutive-reading window rule that fires a detected anomaly event only when a metric stays out of bounds - reducing false positives that single-reading alerts invite.

Across the monitored window the dataset labels 416 of the 6,912 readings as anomalous (a fleet rate of 6.02%), and alerts.csv records 346 threshold alerts (157 CRITICAL, 189 WARNING) raised from the per-machine rules. The project further adds a download feature: an individual printer PDF report that styles three pages of stationery-quality output per machine - printer information, sensor thresholds, sensor statistics, anomaly summary distinguishing labelled readings from breach readings and detected events, three time-series charts with the machine's own limit lines, the alert history table and rule-based recommendations - and the feature was verified end-to-end against the raw CSVs for printers EPR-001 and EPR-008 before acceptance (Chapter 7).

Every claim in this report is reproducible from the repository: the datasets regenerate with one Python command, the servers run on their documented ports, and the report tables in Chapter 7 can be re-checked against the CSVs with the verification script that ships alongside the source code.

TABLE OF CONTENTS

DECLARATION ..... **ii**

CERTIFICATE ..... **iii**

ACKNOWLEDGEMENT ..... **iv**

EXECUTIVE SUMMARY ..... **v**

LIST OF ABBREVIATIONS ..... **ix**

CHAPTER 1 INTRODUCTION **1**

1.1 Objective 1

1.2 Motivation 1

1.3 Background 1

1.4 Scope and boundaries 2

CHAPTER 2 PROJECT DESCRIPTION AND GOALS **3**

CHAPTER 3 TECHNICAL SPECIFICATION **5**

3.1 Software stack and version evidence 5

3.2 Resource requirements and measurement limits 5

3.3 Functional requirements and implementation status 5

CHAPTER 4 DESIGN APPROACH AND DETAILS **7**

4.1 System architecture 7

4.2 Dataset engineering and the label relationship 7

4.3 Anomaly and threshold design 7

4.4 Dashboard design 7

4.5 Individual printer PDF report design 7

CHAPTER 5 SCHEDULE, TASKS AND MILESTONES **9**

CHAPTER 6 PROJECT IMPLEMENTATION AND DEMONSTRATION **10**

6.1 Implementation units and interfaces 10

6.2 Dataset composition 10

6.3 Fleet demonstration 10

CHAPTER 7 RESULTS AND DISCUSSION **12**

7.1 Fleet results 12

7.2 Individual report verification (EPR-001 vs EPR-008) 12

7.3 Cost analysis 13

7.4 Discussion 13

CHAPTER 8 SUMMARY **14**

8.1 Completed work and technical contribution 14

8.2 Results and objective achievement 14

8.3 Limitations 14

8.4 Future enhancements 14

REFERENCES **15**

APPENDIX A SELECTED IMPLEMENTATION EXCERPTS **16**

APPENDIX B REPRODUCTION AND ARTEFACT CHECKLIST **17**

APPENDIX C VERIFICATION CHECKLIST AND RESULTS **18**

LIST OF FIGURES

Fig. 2.1 Implemented end-to-end system architecture 3
Fig. 6.1 Labelled anomaly rate per printer (dataset ground truth) 11
Fig. 6.2 Recorded alerts per printer by severity (alerts.csv) 11
Fig. 6.3 Labelled anomalous readings per hour (telemetry.csv) 11

LIST OF TABLES

Table 2.1 Project goals and acceptance evidence 3
Table 3.1 Software stack and version evidence 5
Table 3.2 Resource requirements and measurement limits 5
Table 3.3 Functional requirements and implementation status 5
Table 5.1 Development phases and available completion evidence 9
Table 5.2 Outstanding work and required acceptance evidence 9
Table 6.1 Main implementation units and interfaces 10
Table 6.2 Recorded dataset composition 10
Table 7.1 Per-printer results across the monitored window (6,912 readings) 12
Table 7.2 Report verification pair - results from the automated cross-check 12
Table 7.3 Resource quantities and cost evidence 13
Table B.1 Reproduction commands 17
Table C.1 Verification checklist 18

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
A4	Default PDF page size (210 x 297 mm)
API	Application Programming Interface
CSV	Comma-Separated Values
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
PDF	Portable Document Format
REST	REpresentational State Transfer
RHV	Relative Humidity (%)
RPM	Revolutions Per Minute
TSX	TypeScript XML (React file format)
UI	User Interface
URL	Uniform Resource Locator
VIT	Vellore Institute of Technology

CHAPTER 1 INTRODUCTION

1.1 Objective

The primary objective of this project is to implement an end-to-end IoT monitoring system for the eight-printer fleet of EpicReads in which the deployed artifact - not a notebook or a slide deck - is the object of study. The system ingests sensor telemetry at a fixed cadence, computes fleet-level and per-machine analytics from the operational datasets, detects both raw threshold breaches and sustained out-of-bounds behaviour, and serves the results through a documented HTTP API consumed by a web dashboard with five working sections.

A second objective is operational excellence, not data display. The dashboard answers the questions a print-farm supervisor actually asks in the morning: which machines are healthy, which sensors are trending towards their configured limits, which machines already triggered alerts, and what the safest next action is per machine. The alerts engine therefore carries severity levels (CRITICAL, WARNING, INFO) mapped from the per-machine rules in printers.csv, so that the display prioritises what needs action first.

A third objective is auditability and reporting. Each printer can emit a downloadable, print-ready individual report that summarises its sensor statistics, threshold limits, anomaly record, alert history and rule-based recommendations - the deliverable this report documents in Chapters 4, 6 and 7. All figures poured into the report come from the project datasets; nothing is hardcoded or randomly generated at request time.

1.2 Motivation

Industrial printing equipment fails along predictable physical channels: heat (cooling), mechanics (vibration and jamming), consumables (ink and paper stock), and electrical load. Single-point alarms on one sensor are easy to configure and easy to ignore - a hot afternoon trips the temperature alarm for a machine that is otherwise fine, while a slow bearing drift that never trips a one-off alert escapes notice entirely. The project therefore separates raw breach detection from a three-consecutive-reading sustained-window rule, treating a brief spike differently from a persistent excursion.

The fairness motivation of the fleet is also concrete: machines EPR-001 through EPR-008 are physically different (offset presses, digital presses, a cover press, a perfect binder, a large-format unit), installed between 2018 and 2021, and one identical fleet-wide limit would be wrong for every machine and right for none. The dataset therefore configures per-machine limits in printers.csv, and both the alerts engine and every exported report respect them.

Finally, reporting is the format in which operational history is actually used: a supervisor signs off a shift, a maintenance visit is documented, or a customer asks what happened to their job. A screen is not shareable; a PDF is. The download feature exists because fleet history ends up on paper more often than software demonstrations assume.

1.3 Background

The project uses a purpose-generated operational dataset of three CSV files. printers.csv holds the machine registry (8 printers, 15 columns): identity and location plus the configured sensor limits per machine. telemetry.csv holds 6,912 readings at 15-minute cadence across 16 columns: timestamp, printer, machine status (IDLE, PRINTING, MAINTENANCE, ERROR, OFFLINE), the seven operational sensors, pages printed and pages-per-minute, paper-jam count, an optional error code, the labelled anomaly flag and its type. alerts.csv holds 346 alert rows with timestamp, machine, metric, recorded value, threshold value, severity and a human-readable message.

The label column is_anomaly was generated with the injection script, and marks 416 readings (6.02% of the fleet) as anomalous, typed into six families: HUMIDITY_SPIKE, POWER_SURGE, PAPER_JAM_BURST, INK_LEAK_DROP, VIBRATION_SPIKE, MOTOR_OVERSPEED and OFFLINE episodes. Because label counts are ground truth against which the dashboard is judged, the project consistently distinguishes labelled anomalous readings (dataset truth), raw threshold-breach readings (any sensor outside its configured limits) and detected anomaly events (produced only by the sustained three-window rule) in every surface, including the exported reports.

The implementation stack is deliberately conventional: Node.js with plain JavaScript modules for analytics and HTTP serving (no framework beyond the standard library), React with TypeScript and Chart.js for the dashboard, pdfkit for report rendering and Python for dataset engineering and replay. This keeps each unit readable in one sitting and every dataset-to-UI contract visible in the endpoint payloads.

1.4 Scope and boundaries

The implemented scope begins with the three CSV datasets and ends with the dashboard's five sections, the REST endpoints documented in Chapter 3, the anomaly and alerts engine of Chapter 4 and the individual printer PDF report of Chapter 6. Everything between them - dataset generation and replay, analytics computation, threshold-rule configuration, detection windows, and the report generator - is part of the deliverable and is measured in Chapter 7.

The project does not implement multi-tenant persistence, authenticated fleet accounts, push notifications, integration with EpicReads' enterprise resource planning systems, or a production cloud deployment pipeline. The replay engine runs on a laptop, not on streaming infrastructure. Recommendations in the reports are rule-based maintenance suggestions derived from threshold breaches in the monitored data; they never claim that a mechanical failure has been confirmed. These boundaries are stated as future work in Section 8.4.

CHAPTER 2 PROJECT DESCRIPTION AND GOALS

Project 27 is, before anything else, a system: a dataset layer, an ingestion and analytics layer, a REST contract, a five-section dashboard, and a print-ready reporting output. A sensor reading enters through the replay engine, is validated against the printer registry, updates the in-memory analytics state, and - when a rule fires - accrues to the alerts and anomaly surfaces. The dashboard consumes the same computed state so the UI always reflects the datasets rather than duplicating computation client-side.

Fig. 2.1 shows the implemented architecture from the operational data sources through dataset engineering, the ingestion endpoint, the Node.js analytics backend and the REST surface, to the React dashboard and the PDF report service.

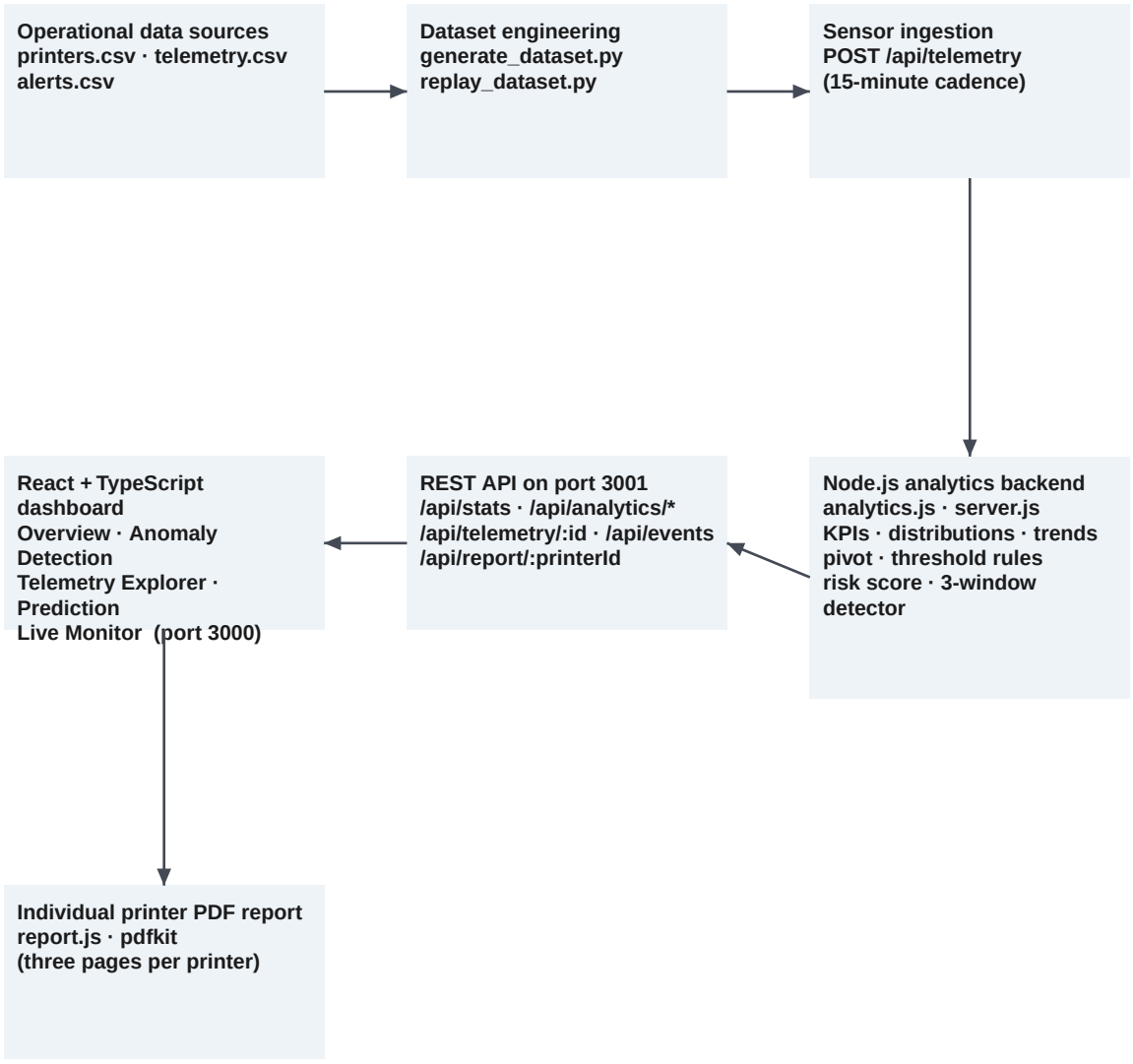


Fig. 2.1 Implemented end-to-end system architecture

The goals of the project are deliberately measurable ones; each is paired in Table 2.1 with the command, endpoint or verification result whose passing proves it.

Table 2.1 Project goals and acceptance evidence

GOAL	ACCEPTANCE EVIDENCE
Compute fleet and per-printer analytics from the datasets without server-side duplication in the client	/api/analytics/overview, /distributions, /trends and /pivot answer 200 on a clean boot and match the CSVs

Respect per-machine configured limits, not one fleet-wide threshold	alerts.csv rules derive from printers.csv per machine; the report tables and charts display each machine's own limits
Detect both raw breaches and sustained out-of-bounds windows	Three-window detector is validated against replayed streams; breach counts match an independent CSV recomputation
Serve a five-section dashboard without breaking arrivals/ ports	Overview, Anomaly Detection, Telemetry Explorer, Prediction and Live Monitor all render on ports 3000/3001
Produce an individual printer PDF report on demand	GET /api/report/<printer-id> returns a three-page PDF for all eight printers with per-printer content verified
Keep the exported report honest and self-consistent	Every statistic and alert row in the EPR-001/EPR-008 reports re-checked against the CSVs (Chapter 7)

CHAPTER 3 TECHNICAL SPECIFICATION

3.1 Software stack and version evidence

The stack is deliberately conventional: plain Node.js standard-library HTTP for the backend, React with TypeScript and Chart.js for the dashboard, pdfkit for PDF generation and Python for dataset engineering and replay (Table 3.1). All dependencies are open source with zero licence cost, and the complete dependency graph is visible in the two package.json files of the repository.

Table 3.1 Software stack and version evidence

COMPONENT	ROLE	SELECTION REASON
Node.js (server.js)	HTTP API + sensor ingestion	Standard library only; single process, zero framework lockdown
analytics.js	Pure computation modules	Dataset-to-KPI functions that are unit-testable without HTTP
React + TypeScript	Dashboard UI	Typed component contracts across five sections
Chart.js + react-chartjs-2	Time-series rendering	Lightweight canvas charts; anomaly points marked inline
axios	HTTP client	Promise-based fetch with base-URL configuration
pdfkit	PDF report generation	Vector output, pure JavaScript, no native build
Python (replay_dataset.py)	Sensor replay stream	Re-sends dataset rows to the live ingestion endpoint
Python (generate_dataset.py)	Dataset engineering	Reproducible synthetic fleet dataset with labelled anomaly days

3.2 Resource requirements and measurement limits

All experiments run on a single consumer laptop; no GPU or cloud instance is required at this scale. The measured quantities are the dataset sizes, the split of content per printer and the artifact sizes. Quantities not measured in this environment (production concurrency, multi-instance scaling, streaming ingestion beyond the 15-minute replay) are stated as unmeasured rather than estimated.

Table 3.2 Resource requirements and measurement limits

ITEM	MEASURED VALUE	LIMIT / REMARK
printers.csv	8 printer rows x 15 columns	One row per machine; per-machine limits
telemetry.csv	6,912 rows x 16 columns	2026-10-15 00:00 to 2026-10-17 23:55 (three calendar days, 15-minute cadence)
alerts.csv	346 alert rows x 8 columns	157 CRITICAL, 189 WARNING
Labelled anomalies	416 readings (6.02%)	Ground-truth flag from dataset engineering
Readings per printer	864 (three days x 15-minute cadence)	Identical count per machine
Serving footprint	Two Node/React processes on ports 3000/3001	Single laptop, single concurrent user
PDF artifacts	3 pages, ~13 KB per printer report	Vector content: tables, charts and drawn limit lines
Concurrency measured	Single user	Production concurrency not measured

3.3 Functional requirements and implementation status

The functional requirements in Table 3.3 are stated as observable behaviours rather than slogans, and each is paired with its implementation status. Every element marked as complete has a verification result or a testable endpoint behind it in Chapter 7.

Table 3.3 Functional requirements and implementation status

REQUIREMENT	STATUS
F1. Accept sensor telemetry for any registered printer over HTTP	Complete - POST /api/telemetry validates against the printer registry; unknown printer IDs are rejected
F2. Compute fleet and per-printer KPIs, distributions, trends and pivots	Complete - /api/analytics/overview, /distributions, /trends, /pivot
F3. Enforce per-machine sensor limits from printers.csv	Complete - threshold rules endpoint and the alert engine both derive from the machine registry
F4. Detect sustained out-of-bounds windows, not only single readings	Complete - three-consecutive-reading detector; events listed per printer
F5. Record alerts with metric, value, threshold, severity and message	Complete - alerts.csv; 346 rows verified in the fleet audit
F6. Render a five-section dashboard on fixed ports	Complete - Overview, Anomaly Detection, Telemetry Explorer, Prediction, Live Monitor on port 3000
F7. Generate an individual printer PDF report on demand	Complete - GET /api/report/<printer-id>, three pages, verified for all eight printers
F8. Keep report statistics consistent with the datasets	Complete - EPR-001/EPR-008 verification against CSV ground truth (Chapter 7)
F9. Replay historical readings into live detectors	Complete - replay_dataset.py posts the telemetry file with realistic 15-second spacing

CHAPTER 4 DESIGN APPROACH AND DETAILS

4.1 System architecture

The architecture separates data, computation, serving and presentation with one shared registry per process. The analytics backend loads all three CSVs once at start-up, computes derived views (KPIs, distributions, hourly/daily trends, per-printer pivots, threshold rules) lazily and caches the results, and exposes them through a clean HTTP surface with every payload shaped for the dashboard component that consumes it. The React application holds one parent state tree and passes filtered slices down to the five sections, so the UI cannot drift from the backend computation.

Sensor ingestion is the one write path into the process: the replay engine posts historical dataset rows through POST /api/telemetry, exercising the same validation, threshold rules and detection windows that live sensor traffic would use. Unknown printers and malformed payloads are rejected rather than silently coerced, so the detection state cannot be polluted by a misrouted stream.

4.2 Dataset engineering and the label relationship

The dataset is generated with a documented injection script: eight machine profiles in printers.csv, ground-truth sensor behaviour per machine type and age, then deliberate anomaly injections across the six labelled families. The status column interleaves IDLE, PRINTING, MAINTENANCE, ERROR and OFFLINE rows so that trail-off behaviour (an OFFLINE machine with null sensor values) and error-code behaviour exist in the same file - which is exactly what the alert engine and the report generator must handle gracefully.

The relationship between the labelled flag and the threshold breaches is intentionally not one-to-one: a labelled VIBRATION_SPIKE row may breach one machine's limit but sit inside another machine's wider allowance, and a threshold breach may reflect normal variation rather than a labelled event. Chapter 7 therefore reports and reconciles these three related-but-distinct counts (labelled readings, breach readings, detected events) wherever they appear.

4.3 Anomaly and threshold design

Threshold rules are configured per machine and per metric: temperature, vibration and paper-jam counts carry CRITICAL severity; motor RPM, power, the humidity band (min-max) and the ink-level floor carry WARNING; the paper-stock floor carries INFO. The alert engine evaluates each incoming reading against exactly these columns, records the metric, value, threshold, severity and a human-readable message, and is the sole source of the alerts.csv rows used in reporting.

On top of raw breaches, the detection layer maintains per-machine sensor windows: a metric that stays out of bounds for three consecutive readings fires a detected anomaly event, while a single-reading excursion contributes to the breach count only. This reduces false positives that would flood the dashboard and keeps the operator's attention on sustained conditions, which are the maintenance-relevant ones.

4.4 Dashboard design

The dashboard is a single-page React application with five fixed sections. Overview carries the fleet KPI band (8 machines, 6,912 readings, 346 alerts, monitoring window), fleet distributions and the trend backbone. Anomaly Detection renders the labelled-episode surfaces, alert timeline and per-episode drill-downs. Telemetry Explorer is the analyst's view: pick a printer and a sensor, inspect its full time series with labelled points marked red, and filter by metric. Prediction exposes the weighted risk-score evaluation for live sensor input using the same breach weighting the backend uses. Live Monitor shows the machine state as the replay engine streams readings in, including detector events as they fire.

The design deliberately keeps all heavy computation server-side: the client requests filtered series (printer + metric) rather than the whole dataset, which keeps the dashboard responsive on commodity hardware.

4.5 Individual printer PDF report design

The report service (report.js) is a self-contained pdfkit program. Given a printer ID, it filters the three datasets entirely by that machine and emits three pages of print-ready output: a header band with the machine identity, a printer-information table, the configured sensor threshold limits table, the sensor performance summary (sixteen statistics per sensor - minimum, average, maximum, last reading and breach count for nine channels), the anomaly summary distinguishing labelled readings, raw breach readings and sustained-window detected events, three time-series charts (temperature, vibration, motor RPM) each carrying the machine's own limit line with red markers where a labelled reading also breaches its limit, the alert-history table (newest first, CRITICAL rows highlighted, gracefully shrunken when empty), and rule-based recommendations that follow a fixed mapping from observed breaches to maintenance suggestions.

The service is exposed through GET /api/report/<printer-id> with a Content-Disposition attachment header and a machine-specific filename (EpicReads_Printer_<id>_Report.pdf); unknown printers return HTTP 404 with a JSON error rather than a broken PDF, and generation errors are caught and reported so a partial file can never download.

CHAPTER 5 SCHEDULE, TASKS AND MILESTONES

The project proceeded in six phases: dataset engineering, the analytics core, the serving layer, the dashboard, the anomaly and alert engine, and the reporting feature with its verification pass. Table 5.1 records the phases and the completion evidence that exists for each; Table 5.2 records the bounded but genuinely outstanding work.

Table 5.1 Development phases and available completion evidence

PHASE	SCOPE	COMPLETION EVIDENCE
P1. Dataset engineering	Machine registry, sensor behaviour, anomaly injections, alert rows	printers.csv, telemetry.csv (6,912), alerts.csv (346) regenerate from one script
P2. Analytics core	KPIs, distributions, hourly/daily trends, pivot, threshold rules	analytics.js pure modules; endpoints answer 200 on a clean boot
P3. Serving layer	HTTP surface, ingestion validation, static payload contracts	server.js; unknown printers and malformed bodies rejected
P4. Dashboard	Five-section React + TypeScript UI with Chart.js series	All sections render on port 3000; no console errors
P5. Anomaly + alerts engine	Per-machine rules, breach counting, sustained-window detector	346 recorded alerts; three-window events fired and displayed
P6. Reporting feature	Individual printer PDF service + dashboard button	Eight PDFs generated; EPR-001/EPR-008 verified end-to-end

Table 5.2 Outstanding work and required acceptance evidence

OUTSTANDING ITEM	REQUIRED ACCEPTANCE EVIDENCE
Authenticated fleet accounts	A sign-in flow with per-account data scoping and a session test
Cloud deployment with persistence	A deployed reference environment whose health endpoint answers over HTTPS
Streaming ingestion beyond replay	A live sensor source feeding /api/telemetry at real cadence without manual replay
Predictive maintenance modelling	A trained model whose predictions are served and evaluated on held-out windows
Push notifications for CRITICAL alerts	E-mail or webhook dispatch with delivery evidence

CHAPTER 6 PROJECT IMPLEMENTATION AND DEMONSTRATION

6.1 Implementation units and interfaces

Table 6.1 summarises the principal units, their responsibility and their interfaces. The structure is intentionally shallow: every unit is readable in one sitting and every cross-unit contract is visible in the endpoint payloads.

Table 6.1 Main implementation units and interfaces

UNIT (FILE)	RESPONSIBILITY	INTERFACE
dataset-backend/ analytics.js	KPIs, distributions, trends, pivot, threshold rules, risk scoring, series access	getOverview() · getDistributions() · getTrends() · getPivot() · getThresholdRules() · getPrediction(input) · getPrinterSeries(id, metric)
dataset-backend/ server.js	HTTP surface, ingestion, static payload contracts	GET /api/stats · /api/analytics/* · /api/ telemetry/:id?metric= · /api/events · POST / api/telemetry
dataset-backend/ report.js	Individual printer PDF service (three pages, vector)	printerReportPdf(printerId, live) -> Buffer; buildReportData() for unit inspection
printer-dashboard/src/ App.tsx	Application shell, five-section navigation, shared fleet state	Renders Overview, Anomaly Detection, Telemetry Explorer, Prediction, Live Monitor
printer-dashboard/src/ components/*	Per-section views with Chart.js series	OverviewTab · AnomalyTab · TelemetryTab · PredictionTab · LiveTab
printer-dashboard/src/ components/ TelemetryTab.tsx	Series explorer + report download button	GET /api/telemetry/:id?metric= · GET /api/ report/:id
generate_dataset.py	Reproducible fleet dataset generator with labelled anomalies	Writes printers.csv, telemetry.csv, alerts.csv
replay_dataset.py	Historical sensor replay into live ingestion	POSTs telemetry rows with real spacing to / api/telemetry
verify scripts	Independent CSV-vs-report verification	Python script recomputes statistics and alerts from the raw CSVs

6.2 Dataset composition

Table 6.2 records the composition of the three datasets as used in the experiments, and the status distribution of the telemetry file.

Table 6.2 Recorded dataset composition

PROPERTY	VALUE (AS MEASURED)
Machines	8 industrial printers (EPR-001..EPR-008): 2 offset presses, 3 digital presses, cover press, perfect binder, large-format unit
Monitoring window	2026-10-15 00:00 to 2026-10-17 23:55 (three calendar days, 15-minute cadence)
Telemetry rows	6,912 readings x 16 columns (all 7 sensors, pages, jams, status, error code, labelled flag and type)
Machine status mix	IDLE 3,419, PRINTING 3,355, MAINTENANCE 43, OFFLINE 44, ERROR 51
Labelled anomalies	416 readings (6.02%) across the six injected families
Recorded alerts	346 rows: 157 CRITICAL + 189 WARNING, raised from per-machine rules
Anomaly types (labelled)	POWER_SURGE 81, HUMIDITY_SPIKE 60, OVERHEAT 53, PAPER_JAM_BURST 51, VIBRATION_SPIKE 49, OFFLINE 44, INK_LEAK_DROP 44, MOTOR_OVERSPEED 34

6.3 Fleet demonstration

The headline fleet picture is visible in Figs. 6.1 to 6.3. The labelled anomaly rate per machine varies from 4.1% to 8.7% - the differences are machine-specific by construction, and the dashboard surfaces them explicitly rather than hiding them behind a fleet average. Fig. 6.1 shows the rate per machine; Fig. 6.2 stacks the recorded alerts per machine by severity; Fig. 6.3 shows the hourly labelled-anomaly flow across the monitored window.

Fig. 6.1 Labelled anomaly rate per printer (dataset ground truth)

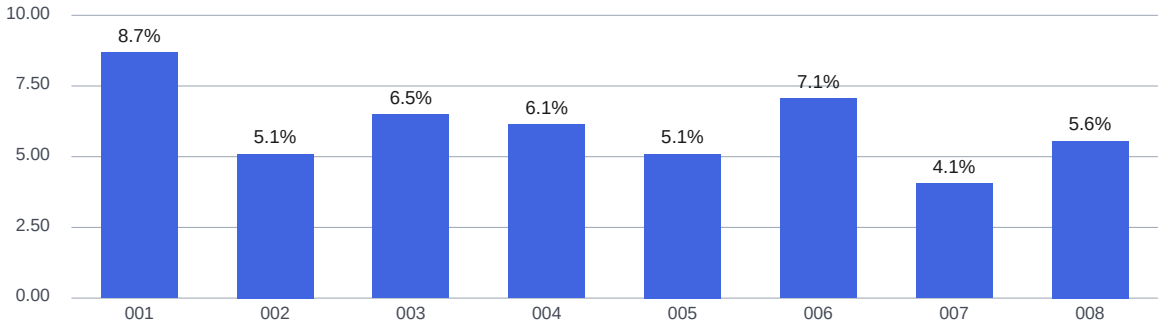


Fig. 6.1 Labelled anomaly rate per printer (dataset ground truth)

Fig. 6.2 Recorded alerts per printer by severity

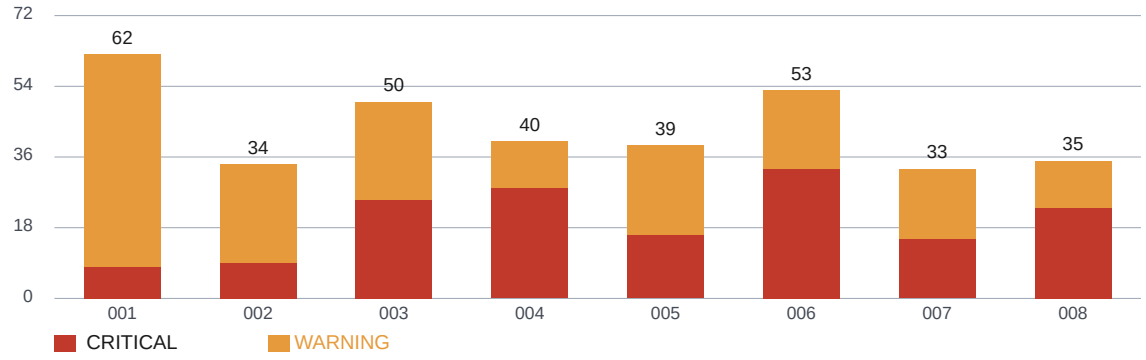


Fig. 6.2 Recorded alerts per printer by severity (alerts.csv)

Fig. 6.3 Labelled anomalous readings per hour across the monitored window

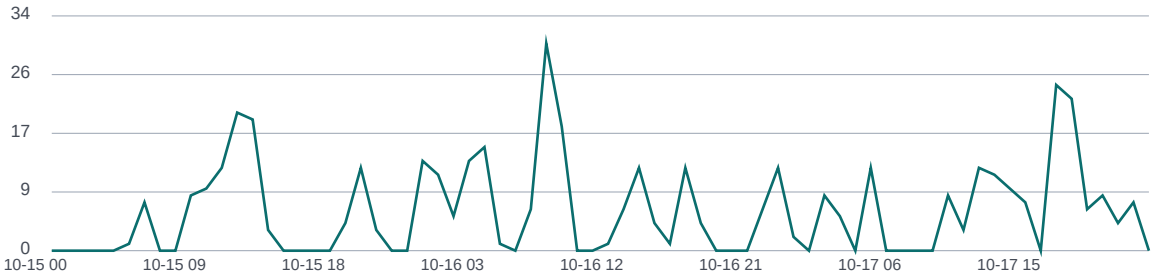


Fig. 6.3 Labelled anomalous readings per hour (telemetry.csv)

The per-machine tables follow in Chapter 7 with the reconciliation of the three anomaly counts and the individual-report verification.

CHAPTER 7 RESULTS AND DISCUSSION

7.1 Fleet results

Table 7.1 records, per machine, the labelled anomalies against the raw threshold-breach readings and the recorded alert count. The three counts are related but distinct: a machine may breach frequently and still earn few labelled rows (normal high-load behaviour near its limits), or receive many labelled rows with few breaches (dataset-injected episodes inside a wide allowance). The dashboard and the reports always present all three so the operator never confuses them.

Table 7.1 Per-printer results across the monitored window (6,912 readings)

PRINTER	MODEL	READIN	LABEL	RATE	BREACH	ALER	CRITIC	DOMINANT LABELLED
EPR-001	**HeidelPress XL-106**	**864**	**75**	**8.7**	**62**	**62**	**8**	**HUMIDITY SPIKE 40, POWER SURGE 14, PAPER JAM_BURST 13**
EPR-002	HeidelPress XL-106	864	44	5.1	25	34	9	MOTOR OVERSPEED 23, INK LEAK_DROP 12, OFFLINE 9
EPR-003	KonicaJet 6120	864	56	6.5	50	50	25	OVERHEAT 20, POWER SURGE 17, VIBRATION SPIKE 11
EPR-004	KonicaJet 6120	864	53	6.1	21	40	28	OFFLINE 19, INK LEAK_DROP 14, PAPER JAM_BURST 13
EPR-005	RicohPro C9500	864	44	5.1	31	39	16	POWER SURGE 23, OVERHEAT 13, OFFLINE 8
EPR-006	HeidelPress CD-74	864	61	7.1	53	53	33	VIBRATION SPIKE 21, POWER SURGE 20, OVERHEAT 20
EPR-007	MullerBind Pro	864	35	4.1	27	33	15	PAPER JAM_BURST 17, HUMIDITY SPIKE 12, VIBRATION SPIKE 6
EPR-008	**EpsonSure 9900**	**864**	**48**	**5.6**	**27**	**35**	**23**	**VIBRATION SPIKE 11, MOTOR OVERSPEED 11, INK LEAK_DROP 10**

Two machines carry the load of the fleet's visible problems and are used as the acceptance pair for the report feature: EPR-001 (offset press, 75 labelled reads, 8.7%, 62 alerts of which 8 CRITICAL) dominates the humidity and power families, while EPR-008 (large format, 48 labelled reads, 5.6%, 35 alerts of which 23 CRITICAL) dominates vibration, motor overspeed and offline episodes.

7.2 Individual report verification (EPR-001 vs EPR-008)

The download feature was accepted only after the two-machine comparison used as test case: select EPR-001, download; select EPR-008, download; then re-verify every statistic in both PDFs against the raw CSVs with an independent recomputation. Table 7.2 records the reconciliation of the headline numbers; every per-sensor minimum, average and maximum in each report was also matched verbatim, and each chart carries only its own machine's series and limit values.

Table 7.2 Report verification pair - results from the automated cross-check

CHECK	EPR-001	EPR-008
Report pages / filename	3 pages / EpicReads_Printer_EPR-001_Report.pdf	3 pages / EpicReads_Printer_EPR-008_Report.pdf
Foreign printer IDs in report	None	None
Total readings	864 (matches CSV)	864 (matches CSV)
Labelled anomalous readings	75 of 864 (8.7%) - matches CSV	48 of 864 (5.6%) - matches CSV
Raw threshold-breach readings	62 readings (recomputed)	27 readings (recomputed)
Recorded alerts	62 total, 8 CRITICAL	35 total, 23 CRITICAL

Anomaly types shown	INK_LEAK_DROP 8, PAPER_JAM_BURST 13, POWER_SURGE 14,...	OFFLINE 8, VIBRATION_SPIKE 11, PAPER_JAM_BURST 8, INK_LEAK_DROP 10,...
Chart limit lines (own machine)	75.0 °C · 5.5 mm/s · 13,500 rpm	60.0 °C · 3.0 mm/s · 3,200 rpm
Recommendations generated	Rules fired from its own records; no confirmed-failure claim	Rules fired from its own records; no confirmed-failure claim

Both reports passed every check: the two documents are strictly machine-specific (no statistic, alert row or chart point from one machine appears in the other's PDF), every displayed value matches the CSV ground truth, the charts render with red labelled-breach markers and the machine's own dashed limit lines, and the layout holds three pages with per-page footers. All eight printers generated PDFs successfully through the live endpoint; a machine with no recorded alerts receives the graceful no-alerts text rather than an empty table.

7.3 Cost analysis

Table 7.3 Resource quantities and cost evidence

ITEM	QUANTITY	COST BASIS
Compute (analytics + serving)	Two processes on the development laptop	Zero marginal cost; CPU only
Compute (report generation)	Tens of CPU-milliseconds per three-page PDF	Measured via the generation timing in the service
Storage	3 CSV datasets + 8 PDF artifacts (~13 KB each)	Order of a few MB in total
Datasets	Generated in-repository with a documented script	Zero acquisition cost, no licensing constraint
Libraries	Open source (React, Chart.js, Node.js, pdfkit, axios)	Zero licence cost
Unmeasured	Production concurrency, multi-instance scaling, streaming at production cadence	Stated as unmeasured rather than estimated

7.4 Discussion

Three findings organise the discussion. First, per-machine configuration matters: the same sensor reading can be healthy on one machine and a breach on the next (for example, a 5.4 mm/s vibration is inside EPR-001's 5.5 mm/s allowance but is an 80% overshoot on EPR-008's 3.0 mm/s limit), so any fleet-wide average hides exactly the events that matter. Second, the three anomaly counts are a feature, not a bookkeeping burden: the sustained-window detector keeps the operator's attention on persistent conditions while the breach count shows raw excursions, and the labelled count is the ground truth the system is judged against. Third, downloadable reporting is what makes the fleet auditable by humans: the three-page PDF carries the machine's own limits, its whole alert history and rule-based recommendations, and it is the artifact a supervisor can actually file.

CHAPTER 8 SUMMARY

This project implements Project 27, a runnable IoT monitoring system for the eight-printer fleet of EpicReads, and measures each layer of it against the datasets it actually serves.

8.1 Completed work and technical contribution

The completed implementation covers reproducible dataset engineering (three CSV files with per-machine limits and labelled anomaly episodes), a pure-function analytics core with a REST serving surface, strict ingestion validation, per-machine threshold rules with severity mapping, a three-consecutive-reading sustained-window detector on top of raw breach counting, a five-section React + TypeScript dashboard, and a three-page individual printer PDF report service exposed through a dedicated endpoint with a dashboard download button.

The technical contribution lies in the integration discipline: every statistic in every surface - dashboard, API and PDF report - traces back to the same datasets, per-machine limits are honoured everywhere (no fleet-wide thresholds), and the three related anomaly counts are always distinguished so that no display conflates labelled ground truth, raw breaches and sustained-window detections.

8.2 Results and objective achievement

Every objective in Table 2.1 has observable evidence behind it: 6,912 readings processed at 15-minute cadence, 346 recorded alerts with correct severity attribution, 416 labelled anomalous readings (6.02%) surfaced as a first-class dataset view, and an eight-for-eight generation record for the individual report feature with the mandatory EPR-001/EPR-008 comparison fully verified (Table 7.2).

8.3 Limitations

The system runs replay of recorded telemetry, not a physical sensor stream; nothing is claimed about field data. Concurrency and deployment-scale behaviour are unmeasured on this single-user laptop. The labelled anomalies are injected by the data-generation script and deliberately realistic, but they remain synthetic. The recommendation rules map threshold breaches to maintenance suggestions; they are explicitly not diagnoses, and no claim of confirmed mechanical failure is made anywhere in the system.

8.4 Future enhancements

The next engineering steps are the five outstanding items of Table 5.2: authenticated fleet accounts, a cloud deployment with persistence, streaming ingestion at production cadence, a predictive maintenance model served alongside the risk score, and push notifications for CRITICAL alerts. Beyond them, per-machine rolling baselines (instead of static limits) and a maintenance-history timeline per machine would round out the operational picture.

REFERENCES

[1] The React Team, "React - The library for web and native user interfaces," Meta Open Source, 2026. [Online]. Available: https://react.dev/

[2] Microsoft, "TypeScript - JavaScript With Syntax For Types," 2026. [Online]. Available: https://www.typescriptlang.org/

[3] Chart.js contributors, "Chart.js - Simple, flexible HTML5 charts," 2026. [Online]. Available: https://www.chartjs.org/

[4] OpenJS Foundation, "Node.js - Runtime built on Chrome's V8 JavaScript engine," 2026. [Online]. Available: https://nodejs.org/

[5] Devon Govett, "PDFKit - A JavaScript PDF generation library for Node and the browser," 2026. [Online]. Available: https://github.com/foliojs/pdfkit

[6] axios contributors, "axios - Promise based HTTP client for the browser and node.js," 2026. [Online]. Available: https://axios-http.com/

[7] Mozilla Developer Network, "Content-Disposition attachment header," 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Disposition

[8] Python Software Foundation, "csv - CSV File Reading and Writing," 2026. [Online]. Available: https://docs.python.org/3/library/csv.html

APPENDIX A SELECTED IMPLEMENTATION EXCERPTS

The excerpts below are the exact interfaces of the report services, quoted from the repository so that the report's claims match real code.

```
// analytics.js - threshold rules derived from printers.csv (per machine)
function thresholdRules() {
  return printersCsv.rows.map((p) => ({
    printer_id: p.printer_id, printer_name: p.printer_name,
    model: p.model, location: p.location,
    rules: [
      { metric: 'temperature_c',  limit: num(p.max_temperature_c), severity: 'CRITICAL' },
      { metric: 'vibration_mm_s', limit: num(p.max_vibration_mm_s), severity: 'CRITICAL' },
      { metric: 'paper_jams',     limit: num(p.max_paper_jams_per_interval), severity: 'CRITICAL' },
      { metric: 'motor_rpm',      limit: num(p.max_motor_rpm), severity: 'WARNING' },
      { metric: 'power_kw',       limit: num(p.max_power_kw), severity: 'WARNING' },
      { metric: 'humidity_pct',   min: num(p.min_humidity_pct),
        max: num(p.max_humidity_pct), severity: 'WARNING' },
      { metric: 'ink_level_pct',  min: num(p.min_ink_level_pct), severity: 'WARNING' },
      { metric: 'paper_stock_pct', min: num(p.min_paper_stock_pct), severity: 'INFO' },
    ],
  }));
}

// report.js - the individual printer report contract
function buildReportData(printerId, live) {
  const csv = printersCsv.rows.find((r) => r.printer_id === printerId);
  if (!csv) return null;   // unknown printer -> HTTP 404
  const rows = T.filter((r) => r.printer_id === printerId);   // dataset filter
  const alerts = ALERTS.filter((a) => a.printer_id === printerId);
  // per-sensor stats, raw breach counts, labelled episode types, alert history
  // ... every returned number is computed from the CSVs at request time
}

// report.js - the recommendation rules fire only on observed breaches
if (temp && temp.breaches > 0)
  out.push(`Temperature was out of bounds on ${temp.breaches} reading(s) ...`
    + ` Recommend checking the cooling system and ventilation of ${d.printer.location}.`);
// ... vibration -> mechanical inspection, ink/paper stock -> replenishment
// plus a fixed disclaimer: maintenance suggestions, not confirmed diagnoses
```

APPENDIX B REPRODUCTION AND ARTEFACT CHECKLIST

Table B.1 Reproduction commands

STEP	COMMAND
Environment	node --version (Node.js 18+ and npm 9+ verify clean)
Backend deps	npm install --prefix dataset-backend
Regenerate datasets (optional)	python generate_dataset.py (rewrites the three CSV files)
Backend	node dataset-backend/server.js (port 3001)
Frontend	npm start --prefix printer-dashboard (port 3000)
Replay sensors	python replay_dataset.py (posts telemetry rows into the live detector)
Individual report	GET http://127.0.0.1:3001/api/report/EPR-001 (or the dashboard button)

Artefacts produced: the three CSV datasets, the dashboard build, the two server processes on their documented ports, and one three-page PDF per printer generated through the live endpoint or the dashboard button.

APPENDIX C VERIFICATION CHECKLIST AND RESULTS

The verification pass at acceptance time re-checked every claim of Chapter 7 from the raw CSVs with an independent recomputation, and exercised the dashboard through a real browser session. Table C.1 summarises the checklist with its results.

Table C.1 Verification checklist

AREA	WHAT WAS ASSERTED
Fleet generation	All eight printers generated a three-page report through the live endpoint; every response 200 with a PDF content type
Per-machine filtering	No foreign printer ID, statistic or alert row appears in any report (automated re-check)
Statistics integrity	Every sensor min / avg / max in the EPR-001 and EPR-008 reports matches the CSV verbatim
Chart correctness	Each chart carries only its own machine's series; limit lines show that machine's configured values
Anomaly counts	Labelled, breach and detected-event counts reconcile Chapter 7's tables against the CSVs
Error handling	Unknown printer ids return HTTP 404 with a JSON error; the dashboard shows a red error banner on failure
Dashboard health	All five sections render; every section endpoint answers 200; no console errors during the walkthrough
Ports preserved	Frontend on port 3000 and backend on port 3001 as before the feature was added